Ahmed Mossad

AI Engineer — Data Science & AI (DSAI)

Giza, Egypt | +20 102 963 8868 | ahmed.abdelfattah.mossad@gmail.com

LinkedIn GitHub Email

Table of Contents

- Summary
- Work Experience
- Graduation Project
- Education
- Technical Skills
- Projects (56 total)
  - A. AI / LLM / RAG
  - B. Big Data & Distributed Systems
  - C. Full-Stack / Software Engineering
  - D. MLOps
  - E. Natural Language Processing / Speech Recognition
  - F. Nature-Inspired Optimization
  - G. Reinforcement Learning
  - H. Deep Learning / Computer Vision
  - I. Classical Machine Learning
  - J. Data Compression / Information Theory
  - K. Systems / Networking
  - L. Data Collection / Miscellaneous
- Certifications
- Activities
- Languages

Summary

AI Engineer with a B.Sc. in Data Science & AI and hands-on FastAPI, PostgreSQL, Redis, and background-job experience from a completed backend AI internship at FlyRank. Built an embeddable lead-capture widget platform with authenticated multi-tenant APIs, rate limiting, spam filtering, async jobs, and a 100%-coverage test suite. Currently a Machine Learning Intern at FlyRank (Oct 2026 – Present) focusing on model training, evaluation, and production integration. Also completed AI Fluency internship at FlyRank and MEAN Stack training at NTI. Strong in RAG/LLM systems (pgvector, LlamaIndex, Groq LLMs), Docker/CI-CD deployment, MLOps, and production-oriented backend architecture.

Work Experience

Machine Learning Intern — FlyRank

Oct 2026 – Present · Chicago, IL, USA · Remote · ML Track

- Assist in developing, training, and evaluating machine learning models using Python, PyTorch, and scikit-learn.
- Prepare, clean, and analyze datasets for ML pipelines; perform feature engineering and data preprocessing.

- Conduct experiments, hyperparameter tuning, and model performance evaluation across multiple algorithms.
- Collaborate with engineering teams to integrate ML models into production services and APIs.
- Research and prototype new ML techniques, document experiments, and maintain reproducible workflows.

Backend AI Engineering Intern — FlyRank

Jul 2026 – Sep 2026 · Chicago, IL, USA · Remote · Backend AI Track

GitHub: flyrank-backend

- Built an embeddable widget and lead-capture platform in FastAPI: tenant-isolated widget CRUD, embeddable versioned JS bundle with immutable HTTP cache headers, and cache-busting on widget update.
- Designed a defense-in-depth public submission pipeline — Pydantic boundary validation, CORS/preflight handling, 3-tier rate limiting (429 + Retry-After), origin checks, honeypot and heuristic spam scoring, and fingerprint dedup.
- Implemented async background jobs with Redis + RQ: IP-to-geo enrichment through a provider fallback chain, fail-open fire-and-forget webhooks, and Groq LLM inference jobs with retry/backoff and idempotency keys.
- Architected a layered router/service/repository backend with Protocol-typed repositories and dependency injection; parameterized SQL with tenant isolation on every query, with SQLite/PostgreSQL/in-memory backends swappable.
- Maintained a 938-test suite at 100% coverage plus a GitHub Actions CI pipeline (isort → black → ruff → pytest) and a Docker Compose stack (FastAPI, PostgreSQL 16, Redis 7).

AI Fluency Intern — FlyRank

Jun 2026 – Jul 2026 · Chicago, IL, USA · Remote · AI Fluency Track

- Completed the FlyRank General AI Fluency track: built a personal AI operating system across Mindset, Practice, Output, and Judgment dimensions of the AI Builder Ladder.
- Designed and documented a repeatable daily AI workflow with clear handoffs between chat, IDE, and automation tools (Claude, ChatGPT, Cursor).
- Shipped a public portfolio artifact demonstrating AI as a core partner — live, verifiable work rather than passive course completion.
- Built a personal quality checklist for evaluating AI output (accuracy, context, tone, completeness) and a failure-mode log with process improvements.
- Completed 5+ assignments and a reviewed capstone; earned verifiable AI Fluency credential with public verification link.

MEAN Stack Web Development Trainee — National Telecommunication Institute (NTI)

Aug 2026 – Sep 2026 · Advanced Summer Training Program 2026 · In collaboration with ITIDA

- Completed the Advanced Summer Training Program 2026, focusing on MEAN Stack Web Development.
- Developed practical full-stack web development skills across frontend, backend, APIs, and database integration.
- Built familiarity with the JavaScript/TypeScript ecosystem and modern web application architecture.

Programming Instructor — GEEK CODE

Jun 2026 – Jul 2026 · Remote

- Deliver one-to-one programming instruction for students aged 12–15 in Python and Scratch, adapting explanations and learning activities to each student's skill level and learning pace.
- Teach core programming concepts including variables, conditionals, loops, functions, problem-solving, and project-based development through interactive sessions.
- Assess student progress through continuous evaluation, practical exercises, and personalized feedback to reinforce understanding and improve performance.
- Contribute to curriculum improvement by providing feedback and recommendations on course content, learning activities, and instructional materials.
- Support students in developing computational thinking, logical reasoning, and programming confidence through structured guidance and hands-on practice.
- Create an engaging and encouraging learning environment that promotes curiosity, creativity, and long-term interest in computer science.

Programming Tutor & Content Creator — Bonyan

2025 – 2026 · Online & On-site · Children's Educational Platform

- Taught programming fundamentals to children across multiple age groups in both classroom and online settings — variables, loops, functions, conditionals, and project-based logic.
- Designed age-appropriate curricula from scratch: broke abstract technical concepts into visual, hands-on activities matched to each learner's level and pace.
- Created engaging mini-projects and coding challenges that maintain student motivation and make progress visible and rewarding.
- Adapted teaching style between younger beginners and more advanced students; patient, clear, and encouraging at every level.
- Developed educational content tailored for children, combining hands-on exercises with engaging explanations to improve learning outcomes.

AI Model Testing Specialist (Freelance) — Outlier

Oct 2024 – Jan 2025 · Remote

- Designed and optimized prompts for large-scale LLMs; built evaluation frameworks to assess and debug model outputs.
- Identified behavioral failure patterns, improving response quality and reliability for enterprise applications.
- Evaluated, ranked, and debugged AI-generated responses.
- Created evaluation frameworks and identified common issues in model responses to improve performance.

Graduation Project

Smart Triangle — AI-Driven Educational Platform

2025 – 2026 · Role: Lead Backend and AI Engine Engineer

Stack: Python, FastAPI, Uvicorn, Pydantic v2, SQLAlchemy (async), asyncpg, PostgreSQL, Supabase, pgvector, LlamaIndex, Groq API (Llama 3.3 70B), sentence-transformers (BAAI/bge-base-en-v1.5), pymupdf4llm, JWT, bcrypt, Google OAuth, OTP (Resend API), React Native, Expo SDK 54, Expo Router v6, TypeScript, axios, expo-secure-store, Docker, Azure Container Apps, GitHub Actions, CI/CD, pytest, pytest-asyncio, ruff, pandas, openpyxl, APScheduler, NumPy, scikit-learn, SSE Streaming, Supabase Realtime (WebSocket), TanStack Query

- Architected a full-stack adaptive learning system with three role-based portals (Student, Parent, Tutor) — 25 features, 42 API endpoints, 17 database tables, deployed to Azure Container Apps via CI/CD pipeline. Zero paid infrastructure ($0/month).
- Built an ETT (Explain The Topic) RAG engine: embeds student queries locally with BAAI/bge-base-en-v1.5, retrieves curriculum chunks via pgvector cosine similarity search, and streams grounded answers via Groq LLM with SSE — end-to-end latency under 2.7 seconds.
- Implemented an Adaptive Exam Engine selecting difficulty per topic based on historical performance; added deep error analysis grounded in curriculum content for each wrong answer.
- Designed a real-time alert system for parents and tutors using Supabase Realtime (WebSocket), with rule-based at-risk detection and Red/Yellow/Green readiness scoring per student.
- Enforced JWT auth, Row-Level Security (RLS), and role-based access at both API and database level; full auth pipeline: JWT refresh tokens, SHA-256 + bcrypt, OTP, Google OAuth.
- Approximately 2,600 lines of async tests across 13 test files; 30+ mobile screens.

Education

B.Sc. in Data Science and Artificial Intelligence Zewail City of Science and Technology — Giza, Egypt Graduated: June 2026

Relevant Coursework: Machine Learning, Deep Learning, NLP, MLOps (DSAI 406), Information Retrieval, Probability & Statistics, Linear Algebra, Calculus I & II, Data Structures & Algorithms, Experimental Design and Data Analysis, Scientific Computing, Numerical Optimization Methods, Advanced Database (CSAI 302), Information Theory (DSAI 325), Reinforcement Learning (DSAI 402), Nature Inspired Computation (DSAI 403), Big Data (DSAI 427), Speech Recognition (DSAI 456), Selected Topics in Deep Learning / GANs (DSAI 490), Linear and Non-linear Mathematics (MATH 303), Senior Graduation Project (CSAI 498/499)

Technical Skills

Category	Skills
Languages	Python, JavaScript/TypeScript, SQL, R, C/C++, Java, Bash
AI / Machine Learning	PyTorch, TensorFlow, Keras, Scikit-learn, HuggingFace Transformers, XGBoost, LightGBM, SHAP, LIME, OpenCV
Generative AI & LLM	RAG Pipelines, LlamaIndex, LangChain, FAISS, pgvector, ChromaDB, Sentence-Transformers, Groq API (Llama 3.3 70B), Google Gemini API, Prompt Engineering, SSE Streaming
MLOps & DevOps	MLflow, DVC, DagsHub, Docker, Docker Compose, GitHub Actions CI/CD, Azure Container Apps, Conda, Sentry, Experiment Tracking, ruff, flake8, black, mypy, pytest
Backend & Databases	FastAPI, Flask, ASP.NET Core, Uvicorn, Pydantic, RESTful APIs, Supabase, PostgreSQL, SQLite, MySQL, SQLAlchemy, Row-Level Security (RLS), JWT, bcrypt, Google OAuth
Frontend & Mobile	React Native, Expo, React, Next.js, TanStack Query, TypeScript, Tailwind CSS, axios
Big Data & Distributed Systems	Apache Spark (PySpark, Spark SQL, Spark MLlib, Structured Streaming), Apache Kafka, Apache Hadoop (HDFS, YARN, MapReduce), Maven
Computer Vision	OpenCV, YOLO (v10/v11/v12), ultralytics, SORT tracking algorithm, Kalman filtering, scikit-image
NLP & Speech	MFCC, FFT/STFT, LPC, Mel spectrograms, GMM, EM algorithm, HMM (Forward algorithm, Viterbi), CTC loss, Bidirectional LSTM, gensim, nltk, hmmlearn, librosa, beam search
Data & Visualization	Pandas, NumPy, Matplotlib, Seaborn, Plotly, ggplot2, EDA, Feature Engineering, Statistical Modeling, Time-Series Analysis, Hypothesis Testing
Optimization & Math	SHAP, LIME, PDPs, counterfactual explanations, metaheuristics (ACO, PSO, GWO, WOA, Firefly, Cuckoo Search, Bat Algorithm, Simulated Annealing, Tabu Search, Hill Climbing), Revised Simplex Method, Normal Equations, Gradient Descent variants
Soft Skills	Problem-Solving, Critical Thinking, Communication, Teamwork, Adaptability, Project-Based Learning, Team Collaboration, Stakeholder Communication, Curriculum Design

Projects

56 projects total, grouped by domain.

A. AI / LLM / RAG

1. Explainable AI (XAI) for Credit Risk Assessment

Date: 2025 · **Language:** Python **Stack:** pandas, numpy, scikit-learn, tensorflow/keras, xgboost, lightgbm, shap, lime, matplotlib, seaborn

GitHub: XAI-Credit-Risk-Assessment

- Designed a comprehensive pipeline for data preprocessing, model training, and performance evaluation on the Lending Club dataset for loan default prediction.
- Implemented and compared Decision Trees, XGBoost, LightGBM, and Deep Neural Networks, achieving high AUC scores.
- Implemented sophisticated explainability techniques to visualize feature importance and model decision-making — SHAP for global feature importance and LIME for local prediction explanations.
- Applied Partial Dependence Plots (PDPs) and counterfactual explanations.
- Conducted comparative analysis of model performance metrics (accuracy, AUC, precision, recall).
- Created interpretable visualizations to communicate complex model behaviors to non-technical audiences.
- Applied state-of-the-art XAI methods to satisfy regulatory requirements for model transparency in financial services.

2. Local RAG-Powered Document Assistant

Date: 2025 · **Language:** Python **Stack:** torch, transformers, sentence_transformers, chromadb, streamlit, tree-sitter, llama_index / LlamaIndex

GitHub: Local-RAG-Document-Assistant

- Designed multi-format document processing system with specialized extraction for different file types.
- Implemented semantic code analysis with structural parsing capabilities for multiple programming languages.
- Created query classification system to route between document and code knowledge bases.
- Developed document summarization with context-aware processing.
- Optimized for low-resource environments (<6GB RAM) with memory management techniques.
- Built comprehensive testing framework for retrieval quality evaluation.
- Offline RAG system for Q&A over documents and codebases without external APIs; CLI and web (Streamlit) interfaces.

---

3. arXiv RAG Web Application

Date: 2024 · **Language:** Python **Stack:** Flask, FAISS, Sentence-Transformers, Google Generative AI (Gemini), PyTorch, pandas, numpy, tqdm

GitHub: Arxiv-RAG-Webapp

- Designed a comprehensive text normalization system optimized for scientific/academic content.
- Engineered prompt templates for effective context integration with LLM responses.
- Implemented GPU acceleration support for improved embedding performance.
- Created a scalable architecture for handling large document collections.
- Developed a responsive web interface for real-time query processing.
- RAG web app for Q&A on arXiv Computer Science papers using FAISS vector search and Google Gemini.

---

4. RAG System with ML Intelligence (CSAI 302 — Advanced Database)

Date: 2024–2025 · **Language:** Python **Stack:** Sentence Transformers (all-MiniLM-L6-v2), ChromaDB, Google Gemini API, XGBoost, scikit-learn, Streamlit, Plotly, Pydantic, Loguru, JSONL feedback system

GitHub: RAG-Optimization-System

- End-to-end RAG system with five-layer pipeline: ingestion, embedding, vector DB, retrieval, generation.
- Included Streamlit UI, feedback system (JSONL), and ML optimization layer with XGBoost-based ParameterOptimizer, DifficultyPredictor, and QualityPredictor.
- Primary contribution: retrieval layer; 40+ engineered features.
- Query latency: 2–3 seconds end-to-end.

---

5. LLM Benchmarking and Optimization Framework

Date: 04/2025 · **Language:** Python **Stack:** PyTorch, HuggingFace Transformers, CUDA, pandas, matplotlib, seaborn, datasets, tqdm

GitHub: LLM-Optimization

- Implemented mixed precision inference, dynamic quantization, and batch size optimization.
- Evaluated on WikiText perplexity and GSM8K accuracy benchmarks.
- Achieved up to 2× throughput improvement via quantization and mixed precision.
- Improved perplexity by over 90% via custom fine-tuning workflows.
- Reduced model inference time while maintaining quality metrics through systematic optimization.
- Created visualization dashboards to compare performance across optimization stages.
- Developed a methodology for optimal batch size selection based on hardware constraints.
- Applied production-ready techniques for deploying more efficient language models.

B. Big Data & Distributed Systems

6. The Digital Librarian (DSAI 427 — Big Data)

Date: 2024–2025 · **Languages:** Java + Python **Stack:** Apache Hadoop 3.3.6, HDFS, YARN, MapReduce, Maven, Docker, Docker Compose, Python, pandas, matplotlib, NumPy, Bash **Role:** Full Java MapReduce, Distributed Cache, Bash scripts, Docker multi-node, Python benchmarking (pair project with Habiba Arafa)

GitHub: Digital-Librarian

- Distributed inverted-index system on Hadoop. Four MapReduce classes: Mapper, Combiner, Reducer, Driver.
- Dataset: 10 Project Gutenberg books (~6MB, ~117K lines).
- Used Distributed Cache for stopword distribution across nodes.
- Wrote Bash automation scripts for cluster setup and job submission.
- Built Python benchmarking suite with Amdahl's Law analysis.
- Results: 39s → 34s → 30s for 1–3 reducers; 40–60% network traffic reduction from Combiner.

---

7. NYC Taxi Trip Duration & Telco Churn Analysis (DSAI 427)

Date: 2024–2025 · **Language:** Python (PySpark) **Stack:** Apache Spark 3.5.8, PySpark, Spark SQL, Spark MLlib, Parquet, pandas, NumPy, Kaggle

GitHub: NYC-Taxi-Spark-Analysis

- Spark analytics on 1.46M NYC taxi records (cleaned to 1.275M).
- Implemented ten analytical queries across RDD, DataFrame, and Spark SQL APIs.
- DataFrames 25–80× faster than RDDs; partition pruning achieved 66× speedup; Parquet 3.96× faster than CSV.
- Second component: Telco churn prediction with Spark MLlib pipeline — Decision Tree, Random Forest, Logistic Regression.

---

8. Real-Time Movie Recommendation System (Big Data Capstone)

Language: Python **Stack:** Apache Kafka 3.5.0, Spark Structured Streaming 3.5.0, PySpark MLlib (ALS), Streamlit, Plotly, Pandas, NumPy, PyArrow, Parquet, WSL Ubuntu

GitHub: Real-Time-Recommendation-System

- Production-grade end-to-end recommendation system using ALS collaborative filtering on MovieLens 1M dataset (RMSE: 0.8705).
- Kafka producer at ~2 events/second with 5% fault injection.
- Sliding-window aggregations (30s window, 10s slide).
- 3-type alert system.
- 5-panel Streamlit dashboard auto-refreshing every 5s.
- p95 end-to-end latency < 3 seconds.

C. Full-Stack / Software Engineering

9. Smart Triangle — Senior Graduation Project

(See Graduation Project section above for full details.)

---

10. Fanni-3lbab: Artisan Service Platform

Date: January 2025 · **Languages:** JavaScript, TypeScript, Python, SQL **Stack:** React, Next.js, Tailwind CSS, Flask, SQLAlchemy, Pydantic, Alembic, MySQL, Node.js, Uvicorn, Google OAuth

GitHub: Fanni-3lbab

- Full-stack platform with role-based authentication (Google OAuth), REST APIs, and database migrations.
- Course landing page built with Next.js; artisan platform built with React.
- Modular React frontend with scalable Flask backend.

---

11. Education Management System

Date: January 2024 · **Languages:** HTML/CSS/JavaScript, C# (ASP.NET Core)

GitHub: Education-Management-System

- Designed and implemented a responsive web application for managing courses, instructors, and student interactions using ASP.NET Core and HTML/CSS/JavaScript.
- Developed dynamic pages for course details, instructor profiles, and student enrollment, ensuring seamless user experience.
- Integrated database functionality to store and retrieve course and user data efficiently.
- Demonstrated strong problem-solving skills by debugging and optimizing application performance.

---

12. Data Visualization and Analysis Web Application

Date: January 2024 · **Language:** Python **Stack:** Flask, Pandas, SQLAlchemy, SQLite

GitHub: Data-Visualization-Web-App

- Built a Flask-based web app to analyze and visualize sales and profit data from a global retail dataset.
- Processed and queried data using Pandas and SQLAlchemy, storing results in an SQLite database.
- Developed RESTful API endpoints to deliver insights on sales trends, customer segmentation, and product performance.
- Enhanced data-driven decision-making by providing interactive and real-time analytics.

D. MLOps

13. End-to-End MLOps Pipeline (Advanced / Standalone)

Date: 2026 · **Language:** Python **Stack:** MLflow, Docker, GitHub Actions, DVC, Conda

GitHub: End-To-End-MLOps-Pipeline

- Built a multi-stage CI/CD pipeline: DVC data pull → MLflow experiment tracking → threshold-based quality gate → automated Docker build triggered only when accuracy ≥ 0.85.
- Implemented reproducible Conda + Docker environments; GitHub Actions workflow with artifact handoff between validation and deployment jobs.
- Mirrors industrial IT automation and reproducibility workflows.

E. Natural Language Processing / Speech Recognition

14. DSAI 456 Speech Recognition — Course Project Series

Date: 2024–2025 (6 Assignments) · ZC-UST **Stack:** Python, PyTorch, librosa, NumPy, SciPy, Matplotlib, Seaborn, hmmlearn, pytest, GitHub Actions, TIMIT, VCTK, MFCC, Mel spectrograms, FFT/STFT, LPC, GMM, EM algorithm, HMM, CTC loss, Bidirectional LSTM, Viterbi algorithm, Forward algorithm, beam search, black, flake8, mypy

GitHub: Speech-Recognition-Pipeline

- **Assignment 1:** RMS/pitch analysis package with CI/CD (GitHub Actions).
- **Assignment 2:** FFT/STFT spectrogram pipeline; frequency-domain analysis.
- **Assignment 3:** Mel spectrogram gender comparison across speaker groups.
- **Assignment 4:** GMM speaker identification — EM algorithm from scratch; VCTK dataset, 110 speakers.
- **Assignment 5:** HMM speaker recognition — custom Forward algorithm and Viterbi decoder in pure NumPy; TIMIT dataset.
- **Assignment 6:** End-to-end phoneme recognizer — 3-layer Bidirectional LSTM (~2M parameters) + custom CTC loss in PyTorch + greedy and beam search decoders; targeting 18–25% Phoneme Error Rate (PER).

---

15. Natural Language Processing: Advanced Word Embedding Models Classification

Date: 05/2025 · **Language:** Python **Stack:** pandas, numpy, nltk, tensorflow/keras, tensorflow_hub, gensim, scikit-learn, matplotlib, seaborn

GitHub: Word-Embedding-Models-Classification

- Engineered complete text preprocessing pipeline including tokenization, stopword removal, and sequence padding.
- Implemented and compared CBOW, Skip-gram, GloVe, and ELMo embedding methods for poem genre classification using biLSTM.
- ELMo contextual embeddings outperform all traditional (static) embedding techniques.

- Conducted thorough model evaluation with confusion matrices and classification reports.
- Created automated analysis tools for misclassification detection and word representation comparison.

---

16. Information Retrieval System Development

Date: May 2024 · **Language:** Python (PyTerrier)

GitHub: Information-Retrieval-System

- Built an Information Retrieval system using Python and PyTerrier to preprocess, index, and retrieve documents from a large dataset.
- Implemented query expansion techniques (e.g., RM3) to improve retrieval accuracy.
- Applied NLP techniques such as tokenization, stemming, and stopword removal for text preprocessing.
- Evaluated retrieval performance using precision and recall metrics.
- Gained hands-on experience in data analysis, NLP, and information retrieval, showcasing problem-solving and research skills.

F. Nature-Inspired Optimization

17. Nature-Inspired Optimization for Sentiment Analysis (DSAI 403 Capstone)

Stack: Python, TensorFlow 2.x, Keras, scikit-learn, NumPy, pandas, LIME, SHAP, matplotlib, seaborn, Jupyter **Metaheuristics:** ACO, PSO, GWO, WOA, Firefly, Cuckoo Search, Bat Algorithm, Simulated Annealing, Tabu Search, Hill Climbing **Model:** Dual-input Conv1D CNN **Dataset:** Sentiment140 (Twitter), 15K samples

GitHub: Nature-Inspired-Computation

- Applied 16 metaheuristic algorithms to tune a dual-input CNN for Twitter sentiment classification.
- Used ACO for feature selection: reduced from 10 to 4 features.
- Tabu Search achieved best baseline at 75.73%; Grey Wolf Optimizer (GWO) achieved best CNN performance at 76.07%.
- Novel application: used metaheuristics to optimize XAI tool parameters — Firefly for LIME, WOA for SHAP.
- Cuckoo Search outperformed Firefly by 9.3%; Bat Algorithm achieved 0.9936 SHAP quality score.
- Overall 25.54 percentage-point improvement over unoptimized baseline.

---

18. Nature-Inspired Optimization Library (DSAI 403)

Stack: Python, NumPy, Pandas, scikit-learn, TF-IDF, RandomForestClassifier, PyTorch, TorchVision, Matplotlib, SciPy, Jupyter, kagglehub

GitHub: Nature-Inspired-Optimization-Library

- Python library implementing 7 metaheuristics from scratch: Hill Climbing, Simulated Annealing, Tabu Search, Bat Algorithm, Cuckoo Search (with Lévy flights), Particle Swarm Optimization (PSO), and Grey Wolf Optimizer (GWO).
- Applied to IMDB sentiment analysis (50K reviews, TF-IDF, RandomForest): accuracy range 85.4%–86.8%.
- Secondary application: PSO for U-Net hyperparameter tuning on CIFAR-10.
- 3,600-line notebook refactored into installable Python package.

G. Reinforcement Learning

19. DSAI 402 Reinforcement Learning — Assignment Series

Date: Fall 2025 (4th Year, 6 Assignments) **Stack:** Python, NumPy, Matplotlib, Seaborn, Gymnasium, ALE-py, OpenCV, Pandas, Jupyter, tqdm

GitHub: RL-Algorithms-From-Scratch *All algorithms implemented from scratch in pure Python/NumPy.*

- **Assignment 1:** Gridworld value functions — Monte Carlo estimation + iterative Bellman equations.
- **Assignment 2:** Tower of Hanoi as MDP + Gymnasium-compliant environment implementation + Policy Iteration.
- **Assignment 3:** Interactive Monte Carlo vs TD(0) animation comparing convergence.
- **Assignment 4:** Breakout (Atari) environment from scratch + full RL zoo: 4 Monte Carlo variants, 5 TD algorithms, 6 Dynamic Programming variants — trained across 2000 episodes.
- **Assignment 5:** Sutton & Barto figure replications + α–n performance heatmap over 100 runs.
- **Assignment 6:** Advanced multi-step methods and eligibility traces.

20. 3D Drone Navigation System with Reinforcement Learning

Date: January 2025 · **Language:** Python **Stack:** NumPy, Matplotlib, mpl_toolkits, Tabulate, Statistics, Logging

GitHub: 3D-Drone-Navigation

- 3D urban drone navigation simulation.
- Implemented Hill Climbing, Simulated Annealing, and Genetic Algorithms for pathfinding.
- Built 3D visualization tools for path rendering using Matplotlib mpl_toolkits.

H. Deep Learning / Computer Vision

21. Person Tracking and Path Visualization System

Date: April 2025 · **Language:** Python **Stack:** ultralytics, opencv-python, numpy, scipy, sort, filterpy

GitHub: YOLO-SORT-Person-Tracker

- Integrated state-of-the-art YOLOv12 object detection with SORT tracking algorithm for reliable person identification and tracking.
- Implemented Kalman filtering for accurate motion prediction in complex scenarios.
- Developed path smoothing algorithms using interpolation for visually coherent trajectory visualization.
- Created a command-line interface with customizable parameters for tracking sensitivity and visual styling.
- Built a robust system capable of maintaining identity tracking through occlusions and camera movement.
- Applicable to surveillance, crowd analysis, sports analytics, retail tracking, and edge/embedded vision systems.

22. Image Enhancement Project for Computer Vision

Date: April 2025 · **Language:** Python **Stack:** OpenCV, NumPy, scikit-image, matplotlib, SciPy

GitHub: Image-Processing-Toolkit

- Implemented robust component extraction algorithms capable of identifying and separating overlapping objects using adaptive thresholding and morphological operations.
- Developed a blurriness correction system comparing multiple sharpening techniques (Unsharp Masking, Laplacian, Wiener Deconvolution) with application-specific parameter tuning.
- Engineered multi-step enhancement chains that outperformed single techniques, balancing detail preservation with artifact reduction.
- Documented comprehensive analysis of technique effectiveness, including pros/cons and potential improvements for each method.
- Applications: medical imaging, document digitization, surveillance.

23. YOLO Object Detection and Tracking Comparison (v10 vs v11 vs v12)

Date: 2023–2024 · **Language:** Python **Stack:** ultralytics, opencv-python, pandas, numpy, matplotlib, seaborn, time

GitHub: YOLO-Model-Comparison

- Built a benchmarking framework comparing YOLOv10n, YOLOv11n, and YOLOv12n for real-time detection and tracking on identical video inputs.
- Measured inference speed (ms), FPS, and tracking efficiency metrics.
- Produced visual and statistical comparisons of model performance; demonstrates expertise across the full YOLO family.

24. Content-Based Image Retrieval with SIFT Features

Language: Python **Stack:** OpenCV (SIFT), NumPy, scikit-learn (K-means, TF-IDF), Matplotlib, tqdm

GitHub: SIFT-Texture-Image-Retrieval

- CBIR system using SIFT feature extraction + Bag of Words vocabulary + TF-IDF weighting.
- Cosine similarity for image-to-image retrieval.
- Conducted experiments on training set size and vocabulary size to analyze their effect on retrieval precision.
- Compared BoW vs TF-IDF weighting schemes.

- Demonstrates rotation-, scale-, and illumination-invariant feature extraction.

---

25. Fourier Transform-Based Image Processing and Filtering

Date: May 2025 · **Language:** Python **Stack:** cv2, numpy, matplotlib, urllib

GitHub: Fourier-Transform-Image-Processing

- Implemented frequency-domain image processing using Discrete Fourier Transform.
- Implemented Ideal, Butterworth, and Gaussian filters for low-pass, high-pass, band-pass, and band-stop operations.
- Created visual before/after comparisons of each filter type.

---

26. Computer Vision: Spatial Domain Image Processing

Date: Spring 2023 (3rd Year, Semester 2) · **Language:** Python **Stack:** OpenCV (cv2), NumPy, Matplotlib

GitHub: Spatial-Domain-Image-Processing

- Implemented complete suite: brightness adjustment, contrast enhancement, histogram equalization, median filtering, sharpening filters, and edge enhancement.
- All implementations include both OpenCV built-ins and custom NumPy equivalents, showing "under the hood" behavior.

---

27. Galaxy Morphology Classification System

Date: January 2025 · **Language:** Python **Stack:** TensorFlow, Keras, NumPy, Matplotlib, h5py, Pandas, scikit-learn **Tooling:** Kaggle (NVIDIA Tesla T4 GPU), Jupyter

GitHub: Galaxy-Morphology-Classification

- Transfer learning with ResNet50V2 and ResNet101V2 on the Galaxy10_DECals dataset.
- GPU-accelerated training with hyperparameter tuning for galaxy morphology type identification.

---

28. Time Series Prediction with Deep Learning Models

Date: Fall 2023 · **Language:** Python **Stack:** TensorFlow/Keras, NumPy, Matplotlib, scikit-learn, keras-tcn

GitHub: Time-Series-Prediction-DL

- Designed a custom data generator to create complex waveforms with configurable noise levels.
- Engineered a sliding window approach for sequence prediction (50 timesteps in, 1 timestep out).
- Implemented advanced TCN architecture with dilated causal convolutions at multiple dilation rates.
- Optimized model hyperparameters including filter sizes, dropout rates, and learning rates.
- Visualized and analyzed prediction accuracy against ground truth values.
- Compared CNN vs TCN architectures; TCN with dilated causal convolutions outperformed CNN.
- Evaluation: MSE and MAE. Used early stopping and dropout regularization.

---

29. Pet Image Denoising with Convolutional Autoencoders

Date: 2023 · **Language:** Python **Stack:** Keras, TensorFlow, NumPy, OpenCV, scikit-learn, Matplotlib, kagglehub **Dataset:** Oxford-IIIT Pet Dataset

GitHub: Pet-Image-Denoising-Autoencoder

- Engineered a complete image processing workflow including resizing, normalization, and noise introduction.
- Implemented a Convolutional Denoising Autoencoder (CDAE) architecture with convolutional, pooling, and upsampling layers.
- Utilized GPU acceleration for efficient model training and evaluation.
- Applied proper train-test splitting methodology for unbiased performance assessment.

---

30. Siamese Neural Network for Fashion Image Similarity Detection

Date: 2023 · **Language:** Python **Stack:** TensorFlow/Keras, NumPy, Matplotlib, scikit-learn **Dataset:** Fashion-MNIST

GitHub: Siamese-Fashion-Similarity

- Implemented Siamese CNN with contrastive loss function.
- Used L1 distance between embeddings for similarity scoring.
- Results: 91.64% accuracy, 92.7% precision, 90.4% recall, 91.5% F1 score.

---

31. Facial Recognition System with Deep Learning Transfer Learning

Date: 2023 · **Language:** Python **Stack:** TensorFlow/Keras, NumPy, Matplotlib, scikit-learn, KaggleHub

GitHub: Facial-Recognition-Transfer-Learning

- Transfer learning with ResNet50 and VGG16 for facial recognition.
- Achieved approximately 79–81% accuracy.

---

32. Flower Classification using Convolutional Neural Networks

Date: 2023 · **Language:** Python **Stack:** TensorFlow/Keras, NumPy, Matplotlib, PIL, ImageDataGenerator

GitHub: Flower-Classification-CNN

- Three custom CNN architectures for flower classification.
- Data augmentation: rotation, width/height shifts, zoom, horizontal flips.
- Best test accuracy: 73.9%.

---

33. Representation Learning with Autoencoders (DSAI 490 — Assignment 1)

Stack: Python 3.10, TensorFlow 2.15, Keras, tf.data, NumPy, scikit-learn, Matplotlib, Pillow, pandas, tqdm, tensorflow-probability, pytest, Conda, Kaggle GPU **Dataset:** Medical MNIST (~58K images, 6 modalities)

GitHub: AutoEncoder-VariationalAutoEncoder

- Implemented three autoencoder architectures: BasicAE (val MSE: 0.00838), DenoisingAE (val MSE: 0.00183, best), CVAE (posterior collapse diagnosed after epoch 2).
- t-SNE and PCA analysis of latent space representations.
- CLI interface and pytest test suite.

---

34. Conditional Date Synthesis with Deep Generative Models (DSAI 490 — Assignment 2)

Stack: Python 3.10, PyTorch 2.5.1, CUDA 11.8, Kaggle GPU **Architectures:** WGAN-GP, Conditional VAE (CVAE), Discrete Diffusion, Transformer Seq2Seq **Techniques:** Gumbel-Softmax, spectral normalization, AdamW, NumPy, Matplotlib, scikit-learn

GitHub: Generative-Date-Synthesis

- Four generative architectures for calendar date synthesis conditioned on day-of-week, month, leap-year, and decade.
- WGAN-GP: best overall (47.58% Condition Pass Rate, 100% month accuracy, 75.63% leap-year accuracy).
- CVAE: 31.48% CPR; Discrete Diffusion: 34.04% CPR; Transformer Seq2Seq: 24.52% CPR (mode collapse observed).

I. Classical Machine Learning

35. Statistical Inference for Diabetes Dataset Analysis

Date: January 2025 · **Language:** R **Stack:** ggplot2, data.table, corrplot, dplyr

GitHub: Statistical-Inference-Diabetes

- Statistical inference, correlation analysis, and comprehensive data visualization on a diabetes dataset.

---

36. Central Limit Theorem Simulation and Analysis

Date: 2025 · **Languages:** R and Python (parallel implementations) **Stack (R):** ggplot2, gridExtra · **Stack (Python):** numpy, matplotlib, seaborn

GitHub: Central-Limit-Theorem-Demo

- Designed parallel implementations in both R and Python, showcasing programming versatility.
- Generated 1,500 random samples across sample sizes n=10, 50, 100.

- Integrated advanced data visualization techniques to illustrate theoretical statistical concepts.
- Performed rigorous statistical analysis validating the Central Limit Theorem for non-normal (exponential) populations.
- Generated comprehensive documentation with visualizations and statistical summaries.

---

37. Health Data Analysis and Prediction Using Machine Learning Models

Date: May 2024 · **Language:** Python **Stack:** Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, TensorFlow, Keras **Tooling:** Jupyter, Google Colab, Kaggle

GitHub: Health-Data-Prediction-ML

- Implemented and compared KNN, SVM, Logistic Regression, Linear Regression, and Neural Networks on a health dataset.

---

38. Titanic Survival Prediction with Naive Bayes Classifier

Date: Spring 2024 · **Language:** Python **Stack:** pandas, numpy, scikit-learn (GaussianNB), StandardScaler, confusion_matrix

GitHub: Titanic-Naive-Bayes

- Engineered meaningful features from raw passenger data: family size, title extraction.
- Implemented systematic data preprocessing workflow to handle missing values.
- Applied proper machine learning validation techniques including train-test splitting and cross-validation.
- Evaluated model performance through accuracy and confusion matrix analysis.
- Demonstrated practical application of the Gaussian Naive Bayes probabilistic classification algorithm.
- Mean cross-validation accuracy: 68.36%.

---

39. Neural Network from Scratch — Logistic Regression for Heart Disease

Date: Spring 2024 · **Language:** Python **Stack:** numpy, pandas, sklearn

GitHub: Neural-Network-From-Scratch

- Implemented logistic regression from scratch: sigmoid activation, binary cross-entropy loss, gradient descent.
- Results: 85.2% accuracy, 87.1% precision, 84.4% recall, 85.7% F1 score.

---

40. Support Vector Machine Clustering Analysis

Date: Spring 2024 · **Language:** Python **Stack:** scikit-learn, pandas, numpy, matplotlib

GitHub: SVM-Clustering-Analysis

- Applied SVM with RBF kernels to 6 clustering benchmark datasets: Aggregation, Compound, Flame, Jain, Pathbased, Spiral.
- Gamma and C hyperparameter tuning experiments across all datasets.

---

41. Linear Regression and Classification from Scratch with Dimensionality Reduction

Date: May 2025 · **Language:** Python **Stack:** numpy, pandas, matplotlib, seaborn, scikit-learn

GitHub: Linear-Regression-From-Scratch

- Implemented Normal Equation linear regression.
- Binary classification on Iris dataset.
- PCA dimensionality reduction (4D → 2D).
- Feature selection and normalization pipeline.

---

42. K-Nearest Neighbors (KNN) from Scratch for Iris Dataset

Date: Spring 2025 · **Language:** Python **Stack:** numpy, pandas, matplotlib, scikit-learn, collections.Counter

GitHub: KNN-From-Scratch

- Custom KNN implementation with Euclidean distance and majority voting.
- Compared K=3, K=5, K=7.
- Achieved 100% accuracy with normalized features.

43. Adam Optimization Algorithm Implementation

Date: January 2025 · **Language:** Python

GitHub: Adam-Optimization-Implementation

- Implemented the Adam optimizer from scratch to minimize a quadratic objective function.
- Parameter tuning (β1, β2, ε, learning rate) and convergence verification.

44. 1D and Multidimensional Optimization Algorithms

Date: January 2025 · **Language:** Python **Stack:** numpy, scipy.optimize, warnings, time

GitHub: Optimization-Algorithms

- Implemented: Fibonacci Search, Golden Section Search, Newton's Method, Quasi-Newton (BFGS), Secant Method, Fletcher-Reeves Conjugate Gradient, Marquardt (Levenberg-Marquardt).
- Tested on Rosenbrock's function and Powell's function.
- Compared against scipy.optimize reference implementations.

45. Revised Simplex Method Implementation

Date: January 2025 · **Language:** Python **Stack:** numpy, scipy.optimize.linprog, dataclasses, typing

GitHub: Revised-Simplex-Method

- Implemented the Revised Simplex Method for linear programming.
- Handles two-phase problems, unbounded solutions, and infeasibility detection.
- Compared against scipy.optimize.linprog as reference.

46. Fuel Consumption Analysis

Date: May 2024 · **Language:** Python **Stack:** Pandas, NumPy, Matplotlib, Seaborn

GitHub: Fuel-Consumption-Analysis

- Conducted data cleaning, validation, and analysis on vehicle fuel consumption datasets to evaluate fuel efficiency and greenhouse gas emissions.
- Leveraged Python libraries (Pandas, NumPy) to process and analyze large datasets, identifying trends and actionable insights.
- Visualized key findings to support data-driven decision-making, showcasing proficiency in exploratory data analysis.

J. Data Compression / Information Theory (DSAI 325 — Java)

47. Adaptive Huffman Coding Algorithm Implementation

Language: Java

GitHub: Adaptive-Huffman-Coding

- Implemented Adaptive Huffman coding with dynamic tree construction.
- Components: Encoder, Decoder, NYT (Not Yet Transmitted) nodes.
- Compression ratio analysis across multiple file types.

48. Color Vector Quantization Image Compression System

Date: April 2025 · **Language:** Java **Stack:** java.util.concurrent, java.io, java.util.logging, java.awt.image

GitHub: Color-Vector-Quantization

- Implemented the LBG (Linde-Buzo-Gray) algorithm for RGB color image vector quantization.
- Multithreading support, early termination, 2×2 pixel block processing.
- Metrics: MSE, PSNR, compression ratio.

49. Vector Quantization Image Compression System

Date: 2023 · **Language:** Java **Stack:** java.awt, javax.imageio, java.io

GitHub: Vector-Quantization-Compression

- LBG + K-means for lossy image compression.
- Configurable block sizes.
- Evaluation: MSE and compression ratio.

---

50. 2-D Feed Backward Predictive Coding for Image Compression

Date: April 2025 · **Language:** Java

GitHub: Predictive-Coding-Compression

- Implemented three predictor types: Order-1, Order-2, and Adaptive (gradient-based).
- 98% reconstruction accuracy.
- Up to 2.667:1 compression ratio.
- Adaptive predictor uses gradient information to handle edges effectively.

---

51. LZ78 Compression Algorithm Implementation

Language: Java **Stack:** java.io, java.util

GitHub: LZ78-Compression

- LZ78 lossless compression with dictionary-based pattern matching.
- Dynamic bit allocation and binary encoding/decoding.

---

52. LZ77 Compression Algorithm Implementation

Languages: Java and Python **Stack (Java):** java.io, java.nio, java.util.regex · **Stack (Python):** re, os

GitHub: LZ77-Compression

- LZ77 with 4096-byte sliding window in both Java and Python.
- Tuple-based encoding (offset, length, next_char).
- CLI interface for compression and decompression.

---

53. Huffman Coding Data Compression System

Date: 2024–2025 · **Language:** Java **Stack:** java.io, java.util (PriorityQueue, HashMap), java.nio.file, BitSet

GitHub: Huffman-Compression

- Complete Huffman implementation with entropy analysis.
- Code table persistence for reuse.
- Binary file handling.

K. Systems / Networking

54. Network File Transfer and Communication System

Date: May 2024 · **Language:** Python **Stack:** socket, struct, time, matplotlib, select

GitHub: Network-File-Transfer-System

- Implemented both UDP and TCP file transfer protocols.
- Retransmission handling and acknowledgment mechanisms for UDP.
- Performance metrics visualization with matplotlib.

L. Data Collection / Miscellaneous

55. YouTube Video Search and Analysis

Date: January 2024 · **Language:** Python **Stack:** serpapi, requests, BeautifulSoup (bs4), html5lib

GitHub: YouTube-Video-Search-Analysis

- YouTube video search via SerpAPI.
- Extract video metadata, identify most frequent channels, download thumbnails.

---

56. Image Featuring & Labeling

Date: January 2024 · **Language:** Python **Stack:** requests, beautifulsoup4, google-search-results, opencv-python, numpy, matplotlib, dlib

GitHub: Image-Featuring-Labeling

- Data acquisition pipeline with web scraping.
- Image processing and facial landmark detection using dlib.

Certifications

- **AI Fluency: Framework & Foundations** — Anthropic Academy (2026) AI fluency and foundations credential for working effectively with large language model systems. Verify certificate
- **AI Fluency Internship** — FlyRank (2026) Completed AI Fluency internship focusing on LLM fundamentals, prompt engineering, and AI system evaluation. Certificate
- **AI Backend Engineering Internship** — FlyRank (2026) Completed AI Backend Engineering internship building production-grade FastAPI backend with LLM inference, RAG pipelines, and async job processing. Certificate

---

Activities

Event Coordinator — Khair ZC, Zewail City *February 2023 – June 2023*

- Led logistics and operations for large charity events.
- Developed stakeholder communication, teamwork, and organizational skills.

Member — Service Union, Zewail City *October 2022 – February 2023*

- Collaborated on campus community projects; demonstrated initiative and responsibility in a team environment.

Languages

- **Arabic:** Native
- **English:** Professional working proficiency

Total projects: 56 (including Smart Triangle listed separately as Graduation Project) · CV template: Jake's Resume (Overleaf / pdfLaTeX)